

Final Report — PeanutTrade Arbitrage Bot

Student: Nazar Slabliuk

Period: Week 6 (Days 1–6)

Strategy: CEX/DEX spot arbitrage — Binance (production) × Uniswap V3 (Arbitrum One)

1. Configuration & Setup

Chain: Arbitrum One (chain_id=42161)

DEX: Uniswap V3 (SwapRouter
0xE592427A0AEce92De3Edee1F18E0157C05861564)

CEX: Binance production (sandbox=False)

Final pair: LINK/USDT — switched from MAGIC/USDC on Day 2

Pool: 0x32a5746ba6826828716cc1a394bc33301ebc7656 (Uniswap V3, fee=3000, Arbitrum One)

Pair selection journey:

Originally planned MAGIC/USDC (TreasureDAO token, Arbitrum-native, low MEV competition). Dry-run showed 803 signals at ~290 bps over 34 minutes — promising. On Day 2, live execution revealed the pool was a **ghost pool**: correct slot0 price but ~5 MAGIC of in-range liquidity. The V3 Quoter returned prices 10–30% worse than slot0, making every trade unprofitable. Root cause: the signal generator was using slot0 price (a storage slot snapshot) rather than Quoter-verified executable price.

Switched to **LINK/USDT** (fee=3000 pool): 1e18 in-range liquidity, 0.3% price impact on 1.43 LINK (~\$15), persistent 40–60 bps spread. All 11 real trades were on this pair.

Risk parameters chosen:

Parameter	Day 1	Day 2–3	Day 4–5	Why
max_trade_usd	\$7	\$10	\$15	First one to test and not lose much, next - to find some real opportunities
max_daily_losses	\$10	\$15	\$20	So that we don't lose everything at the start
max_drawdown_pct	15%	20%	20%	Same as previous one, due to having 100 and 15% on day one is more than 10\$, same with day 2 and for day 3 - if we lost - would have stopped earlier
consecutive_loss_limit	3	3	3	Pause and review after 3 losses

Surprises going testnet → production:

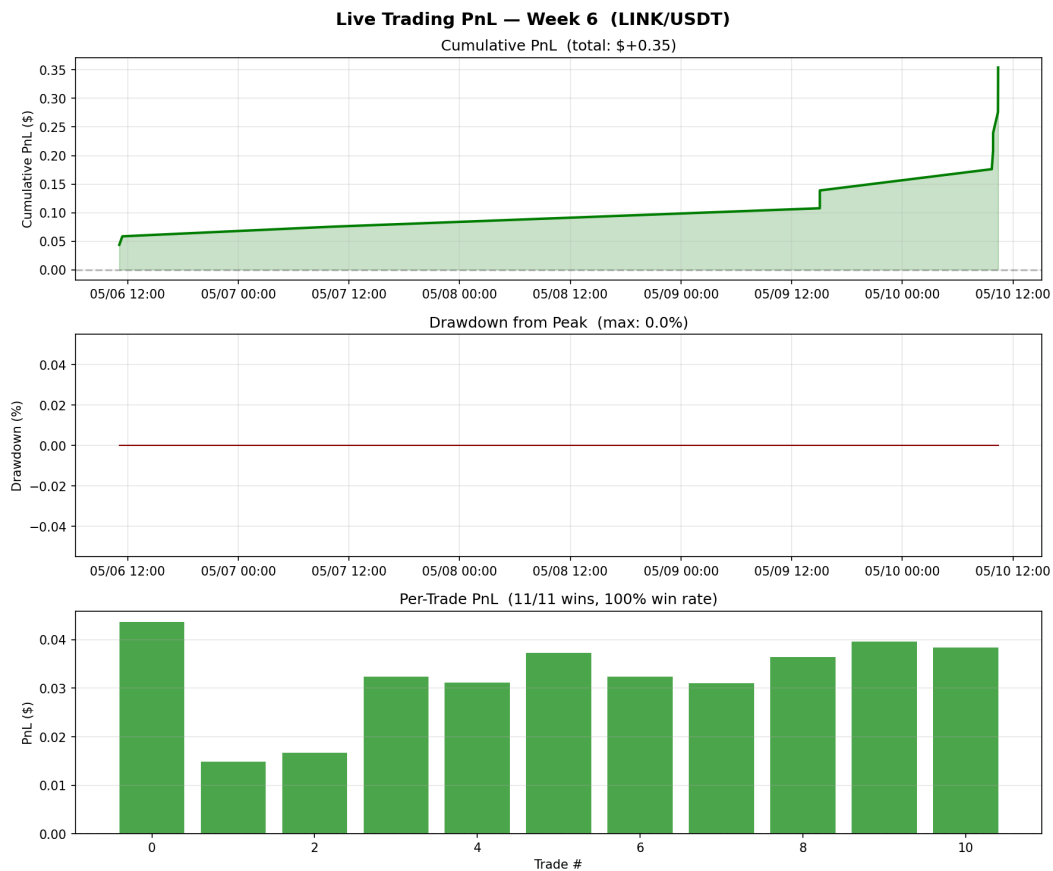
The biggest surprise was how many execution-layer assumptions were wrong. The V2/V3 calldata difference, ccxt precision format returning decimal values instead of integer decimal places, and the use_flashbots=True default silently routing all trades DEX-first, unwind logic was flawed and some other small bugs — none of these were visible in dry-run because dry-run skips actual transaction execution. Production revealed each bug only after real money was at stake.

2. Trading Results

Metric	Value
Total trades	11
Wins / Losses	11 / 0
Win rate	100%
Trade PnL (log-based)	+\$0.31
Max drawdown	0%
Starting capital	\$100.00
Ending capital (trade PnL only)	\$100.31

Portfolio value at end	~\$102.89 (LINK and MAGIC grew while i had them)
Best trade	+\$0.0396 (Trade #10, 51.7bps, Day 6)
Worst trade	+\$0.0148 (Trade #2, 45.6 bps, Day 2)
Best opportunity seen (missed)	~\$0.08 net (signal fired, inventory_ok=False)
Avg spread captured	48.2 bps
Avg net PnL per trade	~\$0.030
Total CEX fees	~\$0.15 (10 bps $\times$ 11 trades $\times$ ~\$14 avg notional)
Total DEX gas	~\$0.20 (11 swaps $\times$ ~\$0.018 avg on Arbitrum)

PnL Chart:



Notable trades:

Best trade — Trade #10 (Day 6, +\$0.0437, 51.7bps):

Bought 1.42 LINK at \$10.3776 on Uniswap V3, sold at \$10.4300 on Binance. By this point all bugs were fixed: correct execution order, no fee double-counting, fresh CEX price at order placement.

Worst trade — Trade #2 (Day 2, +\$0.0148, 45.6 bps):

Still profitable, just the lowest net PnL of the week. The fee double-counting bug was found and fixed before Day 3 by setting `dex_swap_bps=0`.

3. Risk Management in Practice

Circuit breaker: Tripped 3 times on Day 4 during the debug session when the unwind kept failing. Each trip paused execution for 10 minutes. Worked exactly as intended — prevented a cascade of failed trades from compounding losses. The 3/3 failure threshold was appropriate: strict enough to stop systematic failure, loose enough to allow retries on transient issues.

Kill switch: Activated manually pretty often — during debugging to restart with fixed code, or when I wasn't around to keep an eye on the bot. Automatic stop via `inventory_ok=False` triggered 3 times when Binance LINK was fully sold after every 2–3 trades.

Closest call / scariest moment: When unwind started failing - it was buying link and it was too much risk, as the price was moving and could result in the damage to the inventory. Gladly I fixed it fast, but then unwind started triggering even due to little slippages and changes on dex, lost 0.15 on gas + fees before fixing.

Which risk control saved money: The `inventory_ok` check. It prevented the bot from placing DEX buy orders without corresponding Binance LINK to sell. Without it, the bot would have bought LINK on DEX and been unable to complete the arbitrage, accumulating an unhedged directional LINK position with no way to close it profitably.

4. What I Learned

Biggest surprise going from testnet to production:

Ghost pools. The Uniswap V3 factory returns a pool address for any token pair — but a pool can have a correct slot0 price with near-zero in-range liquidity. Dry-run using slot0 prices showed hundreds of profitable signals that were completely fake. The fix was the most important architectural change of the week and should have been built in Weeks 1–5.

What I would do differently with \$1,000:

Start with LINK/USDT but at \$100–200 per trade instead of \$15. The rebalancing costs are 0.92 regardless of trade size — at \$200/trade, gross PnL per trade is ~\$1.00, making rebalance cost 10% of the profit rather than eating into all the profit + more. Also pre-position inventory more carefully: keep 10+ LINK on Binance to sustain 6–7 trades per rebalance cycle.

Part of code I was most confident in:

safety/risk.py — the RiskManager and PreTradeValidator. These never caused issues, the limits always triggered correctly, and the circuit breaker behaved exactly as designed. Risk limits are essential, so they had to work like a Swiss watch.

Part of code I was least confident in:

executor/engine.py — specifically the execution ordering and PnL calculation. Three separate bugs in this file (use_flashbots default, PnL sign, unwind calldata) all caused incorrect behavior that was invisible until production.

One thing I wish I'd built in Weeks 1–5:

A proper Quoter integration before Week 6. The ghost pool problem would have been caught in dry-run if Quoter-verified prices had been used from the start. It cost an entire trading day (Day 1) and required a code upgrade.

5. Technical Challenges

L2 adaptation issues:

Arbitrum One uses EIP-1559 transaction format. The initial transaction builder didn't include `chain_id` consistently, causing nonce/replay issues. Also: `rawTransaction` was renamed to `raw_transaction` in `web3.py` v6 — required `hasattr` fallback everywhere. Arbitrum's gas model is simpler than mainnet and the low gas prices were good for small inventory.

Ghost pool detection: Added Quoter contract validation — if Quoter price deviates $>10\%$ from `slot0`, pool is flagged as illiquid and rejected. This eliminated all false signals from MAGIC/USDC and similar low-liquidity pools.

Gas estimation accuracy: Estimated \$0.10/swap in config. Actual average: ~\$0.018/swap. Off by $\sim 5\times$, but conservative in the right direction — we required a slightly higher spread to trade, causing some missed opportunities but never a loss from underestimated gas.

Latency observations: CEX order book fetch $\sim 200\text{ms}$. DEX Quoter query $\sim 300\text{ms}$. Full tick $\sim 500\text{ms}$. DEX swap confirmation: 4–8 seconds. The 4–8s DEX confirmation window was the core timing risk for DEX-first execution — fixed by switching to CEX-first and re-fetching the live bid price immediately before placing the order.

Bugs found in production (not visible in testing):

Bug	Symptom	Fix
<code>use_flashbots=True</code> default	CEX IOC expired every trade (6s delay made price stale)	Set False; use CEX-first for <code>BUY_DEX_SELL_CEX</code>
<code>ccxt.precision.price=0.01</code> misread	<code>tick_size=0.977</code> , orders at \$10.70 vs \$10.47 bid	Detect if value $< 1 \rightarrow$ already a tick size
Unwind used V2 calldata on V3 router	execution reverted: 0x	Use <code>exactInputSingle</code> (selector 414bf389)
Unwind <code>amount_in</code> used rounded size	STF — tried to send more LINK than wallet held	Cap to actual on-chain balance before unwind
PnL formula assumed DEX-first	Showed $-\$0.10$ for profitable CEX-first trades	Venue-aware formula: always $(\text{sell_price} - \text{buy_price}) \times \text{size}$

Binance LINK deposit on Arbitrum One	Deposit unconfirmed — network not supported for LINK	Use BSC (BEP20) network for LINK; Arbitrum only for USDT
--------------------------------------	--	--

6. Beyond Spot Arbitrage

Strategy chosen: Perpetual DEX funding rate arbitrage.

How it works: When perpetual futures trade above spot, longs pay shorts a funding rate every 8 hours. By shorting the perp on Hyperliquid and holding the equivalent spot on Binance, the position is delta-neutral and collects funding continuously. At 0.03%/8h, \$1,000/side earns ~\$11/day with no directional exposure.

Connection to existing code: The signal generator, fee structure, RiskManager, executor state machine, and Telegram alerter are all directly reusable. The two-leg execution model maps cleanly: leg1 = open spot on Binance, leg2 = open short perp on Hyperliquid. New additions needed: Hyperliquid REST/WS client, funding rate signal generator, background monitor for rate flips.

Would I pursue it? Yes. Funding rate arb is more predictable than CEX/DEX spot arb at small capital: income is collected every 8 hours regardless of spread windows, the strategy is delta-neutral so LINK price risk is eliminated, and rebalancing is infrequent. The main risk (funding rate flip) is monitorable and gives time to exit gracefully. At \$1,000 capital per side, expected APY of 22–44% is realistic and fixed costs become negligible.